

1. 다음 코드의 실행 결과를 쓰시오.

```
1 s = "DATABASE"
2 cols = 4
3 rows = len(s) // cols
4 r = ""
5 for c in range(cols):
6     for row in range(rows):
7         r += s[row * cols + c]
8 print(r)
```

2. 다음 코드의 실행 결과를 쓰시오.

```
1 def f(s):
2     r = ""
3     for ch in s:
4         r += chr((ord(ch) - 65 + 2) % 26 +
5 65)
6     return r
7 def g(s):
8     cols = 2
9     rows = len(s) // cols
10    r = ""
11    for c in range(cols):
12        for row in range(rows):
13            r += s[row * cols + c]
14    return r
15
16 print(g(f("PLANET")))
```

3. 다음 코드의 출력 문자열을, 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

```
1 def f(s):
2     r = ""
3     for ch in s:
4         r += chr((ord(ch) - 65 + 1) % 26 +
5 65)
6     return r
7 def g(s):
8     cols = 3
9     rows = len(s) // cols
10    r = ""
11    for c in range(cols):
12        for row in range(rows):
13            r += s[row * cols + c]
14    return r
15
16 print(g(f("PUZZLE")))
```

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

4. 다음 코드의 실행 결과를 쓰시오.

```

1  def f(s):
2      cols = 2
3      rows = len(s) // cols
4      r = ""
5      for c in range(cols):
6          for row in range(rows):
7              r += s[row * cols + c]
8      return r
9
10 def g(s):
11     r = ""
12     i = 0
13     while i < len(s):
14         c = s[i]
15         cnt = 0
16         while i < len(s) and s[i] == c:
17             cnt += 1
18             i += 1
19         r += c + str(cnt)
20     return r
21
22 print(g(f("AAAABBBB")))

```

5. 다음 코드의 출력 문자열을, 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

```

1  def f(s):
2      key = [1, 0]
3      k = 2
4      r = ""
5      for b in range(0, len(s), k):
6          for j in key:
7              if b + j < len(s):
8                  r += s[b + j]
9      return r
10
11 def g(s):
12     cols = 2
13     rows = len(s) // cols
14     r = ""
15     for c in range(cols):
16         for row in range(rows):
17             r += s[row * cols + c]
18     return r
19
20 print(g(f("COMPUTER")))

```

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

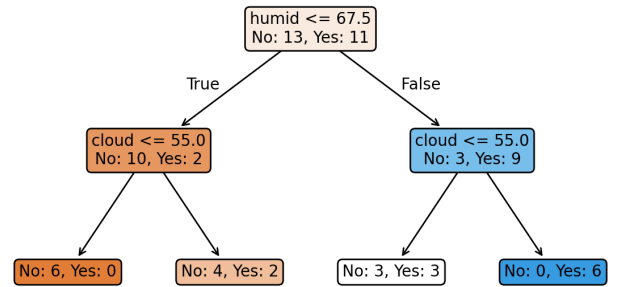
6. 다음 코드의 실행 결과를 쓰시오.

```

1 class Car:
2     def __init__(self):
3         self.speed = 0
4     def up(self, a):
5         self.speed = self.speed + a
6     def stop(self):
7         self.speed = 0
8
9 c = Car()
10 c.up(30)
11 c.up(20)
12 c.stop()
13 c.up(5)
14 print(c.speed)

```

8. 다음 의사결정트리를 보고 물음에 답하십시오. (humid(습도%), cloud(구름량%))



입력 humid=85, cloud=30 가 도달하는 칸(leaf)에 적힌 No 개수와 Yes 개수를 [No, Yes] 순서로 쓰시오.

7. 다음 코드의 실행 결과를 쓰시오.

```

1 class Calc:
2     def __init__(self, x):
3         self.x = x
4     def double(self):
5         self.x = self.x * 2
6
7 c = Calc(5)
8 result = c.double()
9 print(c.x, result)

```

9. 다음 k-Means에 대한 설명을 읽고, 옳으면 O, 틀리면 X로 판정하시오.

보기

- ㄱ. k-Means는 정답(레이블) 없이 데이터를 묶는 비지도 학습이다.
- ㄴ. k는 나눌 군집의 개수이고, 초기 중심점도 k개를 정한다.
- ㄷ. 같은 데이터, 같은 k값으로 학습하면 결과가 동일하다.
- ㄹ. 결과를 고정하려면 random_state(시드)를 지정한다.
- ㅁ. 각 점은 가장 가까운 중심점에 배정되고, 중심점은 배정된 점들의 평균으로 이동한다.
- ㅂ. 반복하면 서로 다른 두 중심점도 결국 같은 위치로 합쳐진다.
- ㅅ. 군집 개수 k는 데이터 개수보다 많아도 된다.
- ㅇ. 결과는 각 점의 군집 번호이며, 번호 자체에 크기/순서 의미는 없다.

10. 다음은 데이터를 학습에 쓰기 전에 정리하는 코드이다.

```
1 import pandas as pd
2
3 df = pd.read_csv('students.csv')
4 df['math'] = df['math'].fillna(df['math'].mean())
5 df = df.dropna()
6 df = df[df['score'] <= 100]
```

2~4번째 줄은 각각 어떤 처리를 하며, 왜 그 처리가 필요한지 서술하시오.

11. 다음 평문을 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

평문: BANANA

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

12. 다음 코드의 실행 결과를 쓰시오.

```

1  def f(s):
2      cols = 3
3      rows = len(s) // cols
4      r = ""
5      for c in range(cols):
6          for row in range(rows):
7              r += s[row * cols + c]
8      return r
9
10 def g(s):
11     r = ""
12     for ch in s:
13         r += chr((ord(ch) - 65 + 1) % 26 +
14 65)
15     return r
16 print(g(f("FRIEND")))

```

13. 다음 코드의 출력 문자열을, 아래 규칙으로 부호화한 최종 비트열을 쓰시오.

```

1  def f(s):
2      r = ""
3      for ch in s:
4          r += chr((ord(ch) - 65 + 2) % 26 +
5 65)
6      return r
7
8  def g(s):
9      cols = 2
10     rows = len(s) // cols
11     r = ""
12     for c in range(cols):
13         for row in range(rows):
14             r += s[row * cols + c]
15     return r
16 print(g(f("GARDEN")))

```

규칙

- ㄱ. 부호화: 빈도가 가장 작은 두 노드를 합쳐 부모 노드를 만든다(반복).
- ㄴ. 빈도가 같으면 ASCII가 작은(빠른) 노드를 먼저 고른다.
- ㄷ. 두 노드를 합칠 때 ASCII가 작은 쪽을 왼쪽(0), 큰 쪽을 오른쪽(1)에 둔다.
- ㄹ. ASCII 순서: A(65) < B(66) < ... < Z(90).

14. 다음 코드의 실행 결과를 쓰시오.

```

1 def f(s):
2     key = [2, 0, 1]
3     k = 3
4     r = ""
5     for b in range(0, len(s), k):
6         for j in key:
7             if b + j < len(s):
8                 r += s[b + j]
9     return r
10
11 def g(s):
12     r = ""
13     i = 0
14     while i < len(s):
15         c = s[i]
16         cnt = 0
17         while i < len(s) and s[i] == c:
18             cnt += 1
19             i += 1
20         r += c + str(cnt)
21     return r
22
23 print(g(f("AABBBBA")))

```

15. 다음 코드의 실행 결과를 쓰시오.

```

1 class Hero:
2     def __init__(self, name, hp):
3         self.name = name
4         self.hp = hp
5
6 a = Hero("기사", 100)
7 print(a.name, a.hp)

```

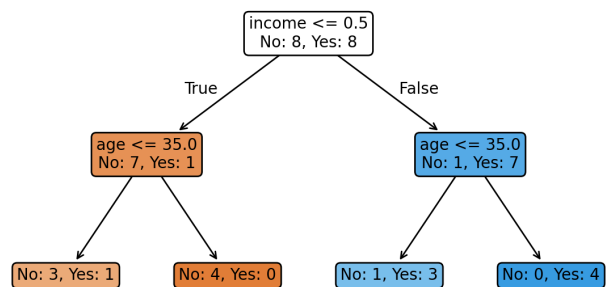
16. 다음 코드의 실행 결과가 '민수'일 때, 빈칸에 들어갈 코드를 쓰시오.

```

1 class Student:
2     def __init__(self, name):
3         _____ = name
4     def hello(self):
5         print(self.name)
6
7 s = Student("민수")
8 s.hello()

```

17. 다음 의사결정트리를 보고 물음에 답하십시오.
(income(소득): 0=낮음 1=높음, age(나이))



입력 income=1, age=25 가 'Yes'로 분류될 확률을 백분율로 쓰시오. (확률 = 도달한 칸의 Yes 개수 / 전체 개수)

20. 지도학습 모델을 만들 때 보통 데이터를 train(학습용)과 test(평가용)로 나눈다. 데이터를 이렇게 나누는 이유를 서술하시오.

1. 정답: DBAATSAE**풀이**

1단계: DATABASE -> DBAATSAE

정답: DBAATSAE

2. 정답: RCGNPV**풀이**

1단계: PLANET -> RNCPGV

2단계: RNCPGV -> RCGNPV

정답: RCGNPV

3. 정답: 10001101100010**풀이**

코드 1단계: PUZZLE -> QVAAMF

코드 2단계: QVAAMF -> QAVMAF

코드 출력: QAVMAF

빈도: A:2, F:1, M:1, Q:1, V:1

F(1) + M(1) -> FM(2): F=왼쪽(0), M=오른쪽(1)

Q(1) + V(1) -> QV(2): Q=왼쪽(0), V=오른쪽(1)

A(2) + FM(2) -> AFM(4): A=왼쪽(0), FM=오른쪽(1)

AFM(4) + QV(2) -> AFMQV(6): AFM=왼쪽(0), QV=오른쪽(1)

코드: A=00, F=010, M=011, Q=10, V=11

비트열: 10 00 11 011 00 010 = 10001101100010

정답: 10001101100010

4. 정답: A2B2A2B2**풀이**

1단계: AAAABBBB -> AABBAABB

2단계: AABBAABB -> A2B2A2B2

정답: A2B2A2B2

5. 정답: 011100110101000010111001**풀이**

코드 1단계: COMPUTER -> OCPMTURE

코드 2단계: OCPMTURE -> OPTRCMUE

코드 출력: OPTRCMUE

빈도: C:1, E:1, M:1, O:1, P:1, R:1, T:1, U:1

C(1) + E(1) -> CE(2): C=왼쪽(0), E=오른쪽(1)

M(1) + O(1) -> MO(2): M=왼쪽(0), O=오른쪽(1)

P(1) + R(1) -> PR(2): P=왼쪽(0), R=오른쪽(1)

T(1) + U(1) -> TU(2): T=왼쪽(0), U=오른쪽(1)

CE(2) + MO(2) -> CEMO(4): CE=왼쪽(0), MO=오른쪽(1)

PR(2) + TU(2) -> PRTU(4): PR=왼쪽(0), TU=오른쪽(1)

CEMO(4) + PRTU(4) -> CEMOPRTU(8): CEMO=왼쪽(0), PRTU=오른쪽(1)

코드: C=000, E=001, M=010, O=011, P=100, R=101, T=110, U=111

비트열: 011 100 110 101 000 010 111 001 =
011100110101000010111001

정답: 011100110101000010111001

6. 정답: 5**풀이**

0 -> +30=30 -> +20=50 -> stop=0 -> +5=5.

정답: 5

7. 정답: 10 None**풀이**double()은 self.x를 2배(10)로 바꾸지만 return이 없어
result는 None.

정답: 10 None

8. 정답: [3, 3]

풀이

입력 humid=85, cloud=30 를 트리 맨 위에서부터 분기 조건(예/아니오)에 따라 내려간다.

도달한 칸: No 3, Yes 3.

정답: [3, 3]

ml-km-002

9. 정답: ㄱ:O, ㄴ:O, ㄷ:X, ㄹ:O, ㅁ:O, ㅂ:X, ㅅ:X, ㅇ:O

채점 포인트

ㄱ O: 정답 레이블 없이 묶으므로 비지도학습.

ㄴ O: k = 군집 수 = 초기 중심점 개수.

ㄷ X: 초기 중심점이 무작위라, 시드를 고정하지 않으면 결과가 달라질 수 있다.

ㄹ O: random_state로 무작위 초기화를 고정하면 결과가 재현된다.

ㅁ O: 배정(가까운 중심) + 재설정(평균 이동)이 k-Means의 핵심 반복.

ㅂ X: 서로 다른 중심점은 각자 영역 점들의 평균으로 갱신되어 다른 위치를 유지한다(같은 점으로 합쳐지지 않음).

ㅅ X: 군집은 데이터 개수보다 많을 수 없다(점 없는 빈 군집 발생, 예러).

ㅇ O: 군집 번호는 단순 식별자라 크기/순서 의미가 없다.

정답: ㄱ:O, ㄴ:O, ㄷ:X, ㄹ:O, ㅁ:O, ㅂ:X, ㅅ:X, ㅇ:O

10. 정답: `fillna(df['math'].mean())`: math 열의 빈 값(결측치)을 그 열의 평균으로 채운다 - 빈 값이 있으면 학습이 안 되므로. `dropna()`: 아직 남아 있는 결측치가 있는 행을 제거한다 - 불완전한 데이터를 빼기 위해. `df[df['score'] <= 100]`: 점수가 100을 넘는 비정상 값(이상치)을 걸러낸다 - 잘못된 데이터가 결과를 왜곡하지 않도록. 즉 결측치와 이상치를 정리해 모델이 올바른 데이터로 학습하게 한다.

채점 포인트

2줄 `fillna(mean)`: 결측치를 평균으로 대체.

3줄 `dropna`: 남은 결측치 행 제거.

4줄 조건 필터: `score>100` 같은 이상치 제거.

공통 이유: 결측치/이상치가 있으면 학습이 안 되거나 결과가 왜곡됨.

g1-065

11. 정답: 100110110

풀이

부호화 대상: BANANA

빈도: A:3, B:1, N:2

B(1) + N(2) -> BN(3): B=왼쪽(0), N=오른쪽(1)

A(3) + BN(3) -> ABN(6): A=왼쪽(0), BN=오른쪽(1)

코드: A=0, B=10, N=11

비트열: 10 0 11 0 11 0 = 100110110

정답: 100110110

12. 정답: GF SOJE**풀이**

1단계: FRIEND -> FERNID

2단계: FERNID -> GF SOJE

정답: GF SOJE

14. 정답: B1A3B2**풀이**

1단계: AABBB A -> BAAABB

2단계: BAAABB -> B1A3B2

정답: B1A3B2

13. 정답: 0111101000000110**풀이**

코드 1단계: GARDEN -> ICTFGP

코드 2단계: ICTFGP -> ITGCFP

코드 출력: ITGCFP

빈도: C:1, F:1, G:1, I:1, P:1, T:1

C(1) + F(1) -> CF(2): C=왼쪽(0), F=오른쪽(1)

G(1) + I(1) -> GI(2): G=왼쪽(0), I=오른쪽(1)

P(1) + T(1) -> PT(2): P=왼쪽(0), T=오른쪽(1)

CF(2) + GI(2) -> CFGI(4): CF=왼쪽(0), GI=오른쪽(1)

CFGI(4) + PT(2) -> CFGIPT(6): CFGI=왼쪽(0), PT=오른쪽(1)

코드: C=000, F=001, G=010, I=011, P=10, T=11

비트열: 011 11 010 000 001 10 =
0111101000000110

정답: 0111101000000110

15. 정답: 기사 100**풀이**

Hero 객체를 만들 때 __init__이 name, hp 속성을 설정한다. a.name=기사, a.hp=100.

정답: 기사 100

16. 정답: self.name

풀이

hello가 self.name을 출력하므로, __init__에서
self.name = name으로 저장해야 한다.

정답: self.name

18. 정답: 가장 가까운 2개의 다수결로 분류하지만, 두 이웃의 클래스가 다르면 1:1 동점이 되어 분류를 정할 수 없다(짝수 k의 동점 문제). 그래서 k는 보통 홀수로 둔다.

채점 포인트

새 데이터에서 가장 가까운 이웃 2개를 찾는다.

그 2개의 클래스를 다수결(투표)한다.

두 이웃의 클래스가 서로 다르면 1:1 동점 -> 다수결로 결정 불가.

결론: 짝수 k(특히 2)는 동점이 생길 수 있어, k는 홀수가 바람직하다.

17. 정답: 75%

풀이

입력 income=1, age=25 가 도달한 leaf: No 1개, Yes 3개 (합 4).

Yes 확률 = $3 / 4 = 75\%$.

정답: 75%

19. 정답: (1) 비지도학습 (정답 y 없이 X만 사용하고 fit_predict로 군집을 찾음). (2) k=3 (점들이 세 무리로 뚜렷이 뭉쳐 있으므로). (3) 전체 데이터가 24개뿐이라 군집을 100개 만들 수 없다. 점이 하나도 없는 빈 군집이 생겨 에러가 나거나, 사실상 점마다 따로 나뉘어 군집화가 무의미해진다.

채점 포인트

(1) X(좌표)만 있고 정답 y가 없으며 fit_predict로 묶음 -> 비지도학습.

(2) 산점도에 점 무리가 3개 -> k=3.

(3) 전체 점이 24개인데 k=100이면 데이터 수보다 군집이 많다.

-> 빈 군집 발생(에러) / 군집화가 의미를 잃음.

20. 정답: 학습에 사용하지 않은 새 데이터(test)로 평가해야, 모델이 처음 보는 데이터에도 잘 작동하는지(일반화) 확인할 수 있다. 학습에 쓴 데이터로만 평가하면 모델이 그 데이터를 외운 것일 수 있어 실제 성능을 제대로 알 수 없다.

채점 포인트

test는 학습에 안 쓴 데이터 -> 새 데이터에 대한 성능 (일반화) 측정.

train으로만 평가하면 외워서 잘 맞는 것과 구분이 안 됨.

'일반화 확인'(또는 새 데이터 성능)이 들어가면 정답.

